Composite

Composite Pattern

Treat tree and leaf alike

DEFINITION

Compose objects into tree structures and let clients treat individual objects and compositions uniformly.

Composite arranges objects into part-whole trees where a Composite (a group) implements the same Component interface as a Leaf, so clients call the root and the operation recurses to every child. Treat a single object and a whole subtree exactly the same way.

WHY IT MATTERS

When data is naturally a tree — files in folders, a UI hierarchy, a menu — client code that branches on "is this a leaf or a group?" duplicates that check everywhere and breaks as the tree deepens. A shared Component interface lets a leaf and a whole subtree be invoked identically; operations become simple recursive delegations. The trade-off: a uniform interface can make the tree overly permissive (leaves inherit child operations they can't honor).

— How it works

Component	The common interface for both leaf and composite (operation()).
Leaf	A primitive with no children — does the actual work.
Composite	Holds children, implements operation() by recursing into them.

HOW TO APPLY

1. Define a Component interface that both leaves and groups implement.
2. Make Leaf do the real work; make Composite hold children and recurse.
3. Call operations on the root; let composites delegate to each child.
4. Decide transparency vs safety: child ops on Component (uniform) or only on Composite (type-safe).

> LITMUS TEST
>
> Is the data a part-whole hierarchy where a node and a subtree should answer the same operations? If clients keep branching on leaf-vs-group, unify them behind a Component.

— Smell → fix

Client code branches on whether each node is a file or a folder:

```
// caller must know if it's a file or folder
if (node.isFolder)
return node.kids.reduce((s,k) => s + size(k), 0)
return node.bytes // both shapes leak to callers
```

↓ ONE INTERFACE FOR LEAF AND TREE

```
interface Component { size(): number }
class File implements Component { // leaf
bytes = 0
size() { return this.bytes }
}
class Folder implements Component { // composite
kids: Component[] = []
size() { return sum(this.kids.map(k => k.size())) }
}
```

File and Folder share Component, so size() works on a single file or a whole tree the same way; Folder just recurses. Add a new node type by implementing Component — client code never branches on shape again.

USE WHEN

- ✓ Part-whole hierarchies (files/folders, UI trees, menus)

AVOID WHEN

- ▲ Flat, non-hierarchical data

— Trade-offs & pitfalls

PROS

+ Uniform client code
+ Recursion is natural

CONS

− Type safety is loosened
− Can over-generalize leaves

COMMON PITFALLS

▲ Transparency vs safety: put add / remove on Component (uniform, but a Leaf must no-op or throw) or only on Composite (type-safe, less uniform). GoF leans transparent.

▲ Deep trees risk stack depth on recursion — an explicit stack or an Iterator helps.

▲ Don't over-generalize: a uniform interface can become a grab-bag of rarely-valid methods.

WHERE YOU SEE IT

- The DOM, React / Vue component trees, and file systems (folders contain files and folders).
- Menus, org charts, scene graphs, and arithmetic expression trees.
- java.awt Containers that hold Components and are themselves Components.

SEE ALSO

Decorator	a Composite with exactly one child that adds behavior instead of aggregating many
Iterator	the usual way to walk the tree a Composite builds
Visitor	applies one operation across the whole tree without bloating the nodes
Interpreter	its AST is a Composite of expression nodes

Interview & sources

Transparency vs safety?

Transparent puts add / remove on Component so all nodes look alike (leaves may fail them); safe puts them only on Composite (type-safe, less uniform). GoF favors transparency.

Composite vs Decorator?

Same recursive structure, but a Decorator is a Composite with a single child whose job is to add behavior, not to aggregate many children.

How do leaves handle add()?

In the transparent design they implement it as a no-op or throw, since a leaf has no children to add to.

Where do you see it?

The DOM, UI component trees, file systems, menus, org charts — anything part-whole where a node and a subtree share an interface.

REMEMBER

Build a part-whole tree where a group implements the same interface as a leaf — clients treat one and many alike.

SOURCES

GoF — "Design Patterns" (1994): Composite (structural)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Iterator & Composite